INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

INGENIERÍA EN SISTEMAS COMPUTACIONALES

SISTEMAS DISTRIBUIDOS

Semestre 2026-2

Práctica 7

PWA con Service Worker



ALUMNO: IVÁN MIRANDA HERNÁNDEZ

PROFESOR: CHADWICK CARRETO ARELLANO

GRUPO: 7CM1

Antecedentes

Servicios Web (Web Services) y Protocolos: Estándar de comunicación distribuida que permite el intercambio de datos entre aplicaciones sobre la red. Garantiza la interoperabilidad entre diferentes lenguajes de programación y plataformas operativas mediante contratos de comunicación claros basados en el protocolo HTTP y serialización en formato JSON.

Arquitectura Cliente-Servidor y Desacoplamiento: Separación de responsabilidades donde el *Frontend* (interfaz de usuario) gestiona únicamente la capa de presentación y el envío de peticiones asíncronas, mientras que el *Backend* (lógica de negocio) procesa las solicitudes y almacena el estado de los datos, logrando un desacoplamiento total entre tecnologías.

Arquitectura REST (Representational State Transfer): Estilo arquitectónico para sistemas distribuidos basado en recursos web que utiliza los métodos nativos de HTTP:

- **GET:** Consulta y recuperación de recursos sin alterar el estado del servidor.
- **POST:** Envío de cargas de datos (*payloads*) para la creación de nuevos registros.
- **PUT:** Modificación y actualización del estado de recursos existentes (ej. cambiar disponibilidad de un libro).
- **Statelessness (Sin estado):** Cada petición contiene toda la información necesaria para ser procesada de forma independiente, eliminando el almacenamiento de sesiones de usuario en el servidor.

Políticas de CORS (Cross-Origin Resource Sharing): Mecanismo de seguridad ejecutado por los navegadores (*Same-Origin Policy*) que restringe peticiones entre dominios o puertos distintos. Se resuelve mediante la inyección explícita de cabeceras HTTP (Access-Control-Allow-Origin: *) en los microservicios backend.

Progressive Web Apps (PWA): Aplicaciones web que emplean APIs modernas de navegador para ofrecer una experiencia similar a una aplicación nativa. Se sustentan en dos pilares fundamentales:

- **Web App Manifest (manifest.json):** Archivo de configuración JSON que define el nombre, íconos, colores de tema y el modo de visualización independiente (standalone) para permitir su instalación en el sistema operativo.
- **Service Workers (sw.js):** Scripts ejecutados en segundo plano por el navegador, independientes de la página web, que interceptan peticiones de red

y gestionan estrategias de almacenamiento en caché para habilitar el funcionamiento *offline*.

Problemática

Migración hacia Aplicación Instalable y Resiliente: Necesidad de evolucionar el cliente web tradicional de React hacia una Progressive Web App capaz de instalarse en el dispositivo del usuario y operar sin depender de una conexión constante al servidor.

Disponibilidad de Datos en Modo Offline: Ante una caída de red o detención de los microservicios Java, la interfaz tradicional perdía acceso a los catálogos, requiriendo mecanismos de caché de interfaz y persistencia local de datos.

Orquestación de Microservicios Independientes: Coordinación en tiempo real entre el Padrón de Alumnos (:8001), Catálogo de Libros (:6001) y el Orquestador (:8080) manteniendo políticas de origen cruzado (CORS) válidas tanto en modo desarrollo como en entorno de producción compilado (vite preview).

Despliegue y Ejecución en Entornos WSL/Ubuntu: Compilación simultánea de procesos Java en segundo plano y generación del empaquetado de producción del frontend (dist/) requerido para el registro válido del Service Worker.Solución

Solución

Diseño e implementación de una arquitectura distribuida de microservicios en Java Nativo conectada a un cliente **Progressive Web App (PWA)** en React:

- **Backend — Nodo 1: Microservicio Alumnos (:8001):** Servidor HTTP que gestiona el padrón de estudiantes. Atiende consultas (GET) y registros (POST).
- **Backend — Nodo 2: Microservicio Libros (:6001):** Servidor HTTP que administra el catálogo y disponibilidad de los libros. Atiende consultas (GET), altas (POST) y reservas/devoluciones (PUT).
- **Backend — Nodo 3: Microservicio Orquestador (:8080):** Punto de acceso principal (*API Gateway*). Recibe las solicitudes del cliente, realiza peticiones HTTP secundarias mediante HttpClient a los Nodos 1 y 2 para validar la transacción, y retorna la respuesta consolidada.
- **Frontend — Cliente PWA React (:4173 / :5173):** Aplicación web progresiva equipada con manifest.json, Service Worker con estrategia *Network-First* con respaldo en caché, almacenamiento en localStorage para consulta de catálogos sin conexión, detección automática del estado de red (online/offline) y botón nativo de instalación (beforeinstallprompt).

Estrategia de Implementación

1. Servidores HTTP Nativos en Java y CORS

Construidos mediante la librería estándar com.sun.net.httpserver.HttpServer con cabeceras de acceso habilitadas para consumir desde el cliente PWA:

exchange.getResponseHeaders().add("Access-Control-Allow-Origin", "*");

exchange.getResponseHeaders().add("Access-Control-Allow-Methods", "GET, POST, PUT, OPTIONS");**2. Automatización del Despliegue (run_backend.sh)**

2. Manifiesto Web e Identidad Instalable (manifest.json)

Configuración de metadatos para que el navegador reconozca la aplicación como instalable en ventana independiente (standalone):

```
{
  "short_name": "BibliotecaREST",
  "name": "Sistema de Biblioteca Distribuidora PWA",
  "start_url": "/",
  "display": "standalone",
  "theme_color": "#0284c7"
}
```

3. Service Worker y Estrategia Offline (sw.js y main.jsx)

Registro del ciclo de vida del Service Worker para almacenar los recursos estáticos (HTML, JS, CSS, SVG) e interceptar peticiones GET mediante caché local cuando no hay red:

```
// Registro en main.jsx y respaldo en caché en sw.js
navigator.serviceWorker.register('/sw.js');
event.respondWith(
  fetch(event.request).catch(() => caches.match(event.request))
);
```

4. Sincronización Online/Offline y Respaldo Local (App.jsx)

Detección dinámica de conectividad (navigator.onLine) para habilitar/deshabilitar transacciones de escritura y recuperar los últimos catálogos desde localStorage al perder conexión:

```
const cacheL = JSON.parse(localStorage.getItem('cache_libros') || '[]');

setLibros(cacheL);

registrarLog("Modo Offline: Cargando catálogos desde almacenamiento local (PWA).", "SISTEMA");
```

Guía de Ejecución

Requisitos Previos

- Java JDK 11 o superior instalado.
- Node.js y npm configurados.
- Entorno de ejecución Linux / Ubuntu / WSL.

Pasos para Levantar el Sistema

1. **Clonar/Navegar al directorio del proyecto:** Abrir la terminal en la raíz donde se encuentran las carpetas backend y frontend.
2. **Ejecutar los Microservicios Backend (Java):** Entrar a la carpeta backend, dar permisos de ejecución al script y correrlo:

```
cd backend

chmod +x run_backend.sh

./run_backend.sh
```

El script compilará las clases de Java y levantará automáticamente los tres servidores en los puertos 8001, 6001 y 8080.

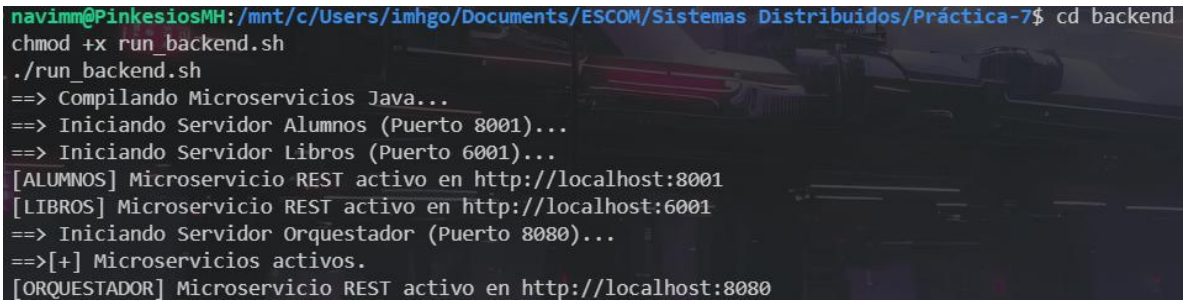
Compilar y Ejecutar la PWA Frontend (React): En una segunda ventana de la terminal, ingresar a la carpeta del frontend, generar la compilación de producción (necesaria para el Service Worker) y levantar el servidor de vista previa:

```
cd frontend
npm install
```

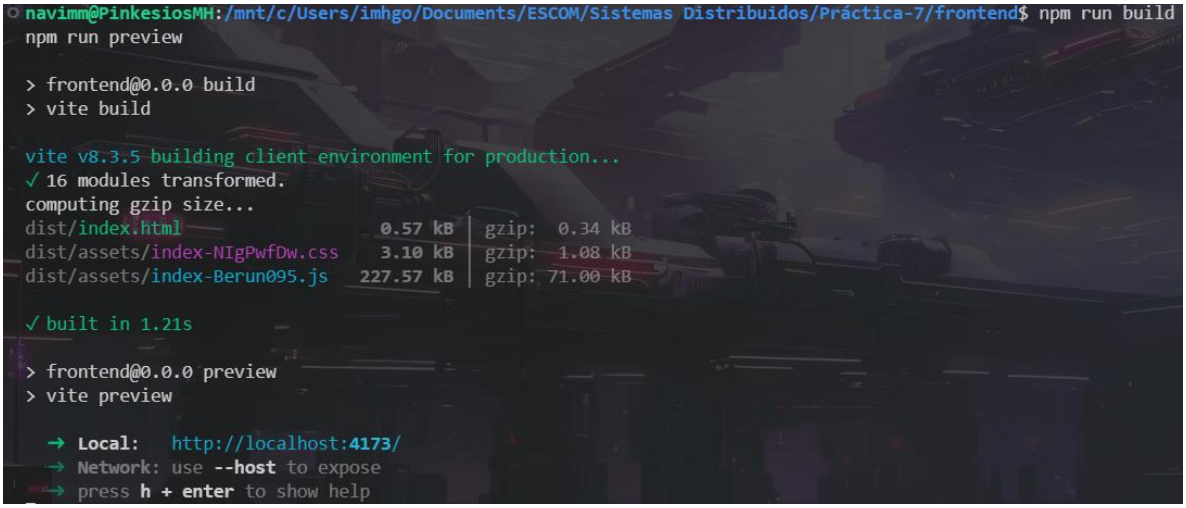
npm run build
npm run preview

1. **Acceso e Instalación en el Navegador:** Abrir Google Chrome o Microsoft Edge e ingresar a http://localhost:4173. Desde la barra de direcciones o mediante el botón **"Instalar Aplicación (PWA)"** se puede instalar en el sistema operativo.

Terminal 1 en backend:



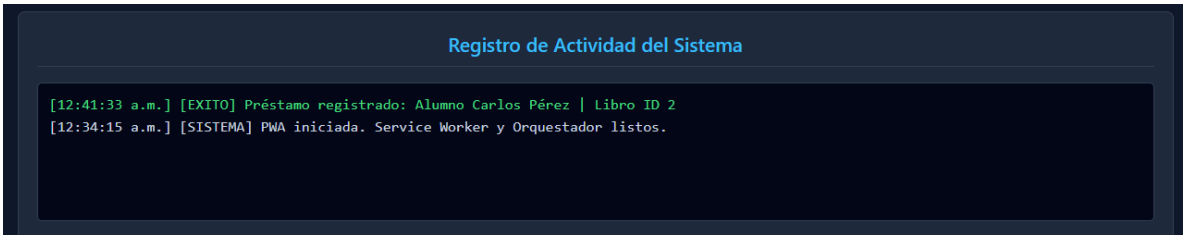
Terminal 2 en frontend:



Resultados

1. Operación En Línea y Transacciones Distribuidas

Con los microservicios activos, la PWA muestra el indicador EN LÍNEA, carga los catálogos en tiempo real y permite registrar préstamos, altas de alumnos/libros y devoluciones, registrando cada evento en la consola de actividad.



2. Funcionamiento en Modo Offline (Resiliencia de Caché)

Al activar la casilla Offline en DevTools (o detener los servidores backend), la PWA detecta automáticamente la desconexión, cambia el indicador a MODO OFFLINE (CACHÉ), bloquea las operaciones de escritura para evitar errores de red y mantiene visibles los catálogos almacenados previamente.

3. Aplicación Instalada en Modo Standalone

Tras ejecutar la instalación, la biblioteca se ejecuta como una aplicación independiente del sistema operativo, sin barra de direcciones de navegador y con acceso directo propio.

Conclusión

- Evolución hacia Aplicaciones Web Modernas: La integración de un Web App Manifest y un Service Worker transformó una SPA convencional en una Progressive Web App instalable, cerrando la brecha entre aplicaciones web y de escritorio.
- Resiliencia mediante Estrategias de Caché: El uso combinado del Service Worker para activos estáticos y localStorage para datos JSON permitió mantener la disponibilidad de consulta del sistema incluso ante interrupciones totales de red.
- Desacoplamiento e Interoperabilidad Total: Se comprobó que una arquitectura RESTful *stateless* en Java nativo puede alimentar sin modificaciones a clientes avanzados con capacidades offline y multiplataforma.